

UNA TORRE DE CONTROL PARA AGENTES

Un agente escribiendo código ya funciona. Seis a la vez, sobre el mismo repo, no. Control Tower es el que reparte el trabajo, reserva el terreno y —lo importante— se niega a decirte que algo salió bien cuando no lo puede demostrar.

4 comandos · 3 hooks · 11 skills forkados · el estado vive en GitHub

EL PROBLEMA

EL CUELLO DE BOTELLA NO ES ESCRIBIR CÓDIGO

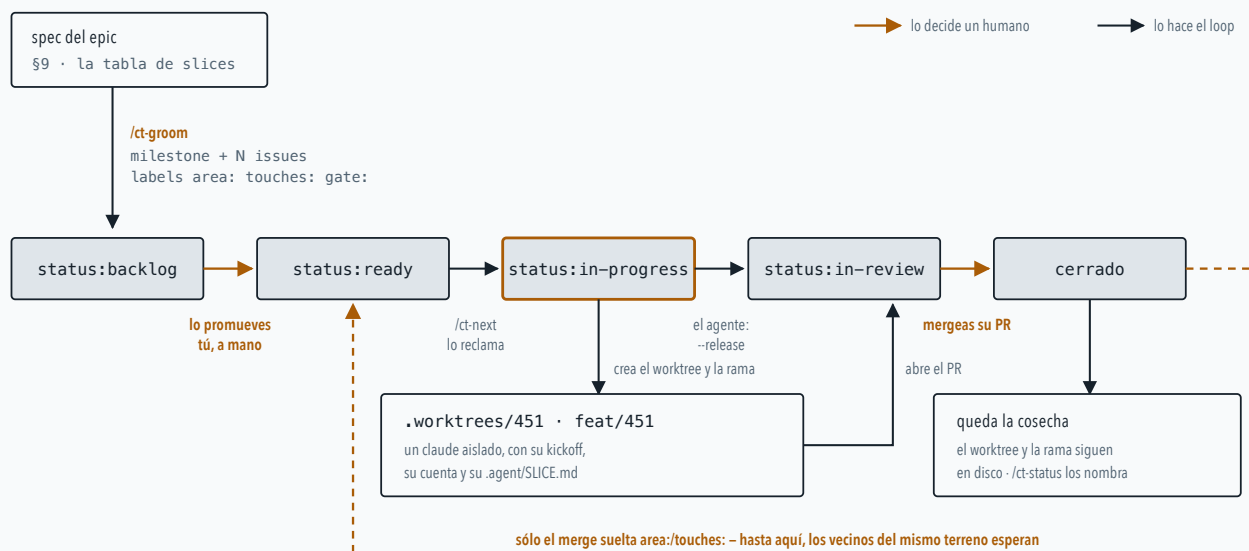
Si lanzas varios agentes contra el mismo repositorio, todos empiezan por hacer lo mismo: coger el trabajo que ya cogió otro, tocar el fichero que otro está reescribiendo, y terminar sin que tú sepas cuál sigue vivo y cuál murió hace tres horas dejando su rama a medias.

Control Tower no escribe código. Se ocupa de lo otro: **qué se despacha ahora, quién tiene reservado qué terreno, y qué ha quedado tirado por ahí.** Es literalmente una torre de control: no pilota los aviones, decide quién despega y cuándo.

EL CIRCUITO

UNA VUELTA COMPLETA, DE LA TABLA AL MERGE

Todo el trabajo se corta en *slices*: filas de una tabla dentro del spec del epic. Cada fila acaba siendo un issue de GitHub, y el issue es el que va cambiando de estado hasta cerrarse. El estado del sistema no vive en ningún fichero local — vive en las labels de esos issues.



El issue es la máquina de estados. Las flechas ámbar son las dos decisiones que siguen siendo tuyas — promover un slice a la cola y mergear su PR—; las oscuras las ejecuta el loop. Que `/ct-groom` cree todo en `status:backlog` y no en `status:ready` es deliberado: nada se despacha sin que un humano lo apruebe.

LAS PIEZAS

CUATRO COMANDOS, Y SÓLO UNO DE ELLOS ESCRIBE

/ct-init

PREPARAR EL REPO

Deja el andamiaje: `.agent/STATE.md`, un `AGENTS.md` con el contrato de la tabla §9, y las líneas del `.gitignore` que hacen falta.

Si el repo ya traía su propio protocolo de claim o su propia ruta de worktrees, **avisa en vez de pisarlo**: elegir cuál manda es decisión tuya, no efecto colateral.

/ct-groom

SPEC → GITHUB

Lee la tabla de slices y la vuelca: milestone, un issue por fila, labels de área y de tipo, gates, y las dependencias `merge-after` entre slices.

Es idempotente por existencia: volver a correrlo no duplica nada — pero tampoco converge solo. Si cambias un título en el spec, **te reporta la diferencia; no la aplica**.

/ct-next

EL DESPACHADOR

Elige el siguiente slice cuyas dependencias estén mergeadas y cuyo terreno esté libre, reclama el issue, crea el worktree y la rama, y arranca un `claude` dentro con su kickoff.

Cuando no despacha nada, **eso es su resultado, no un error**: te dice exactamente qué lo impide y con qué comando se arregla.

/ct-status

SÓLO LEE

Responde de una vez las tres preguntas: qué está en vuelo, qué ha entregado, qué es residuo. Cruza issues, worktrees, ramas y procesos vivos.

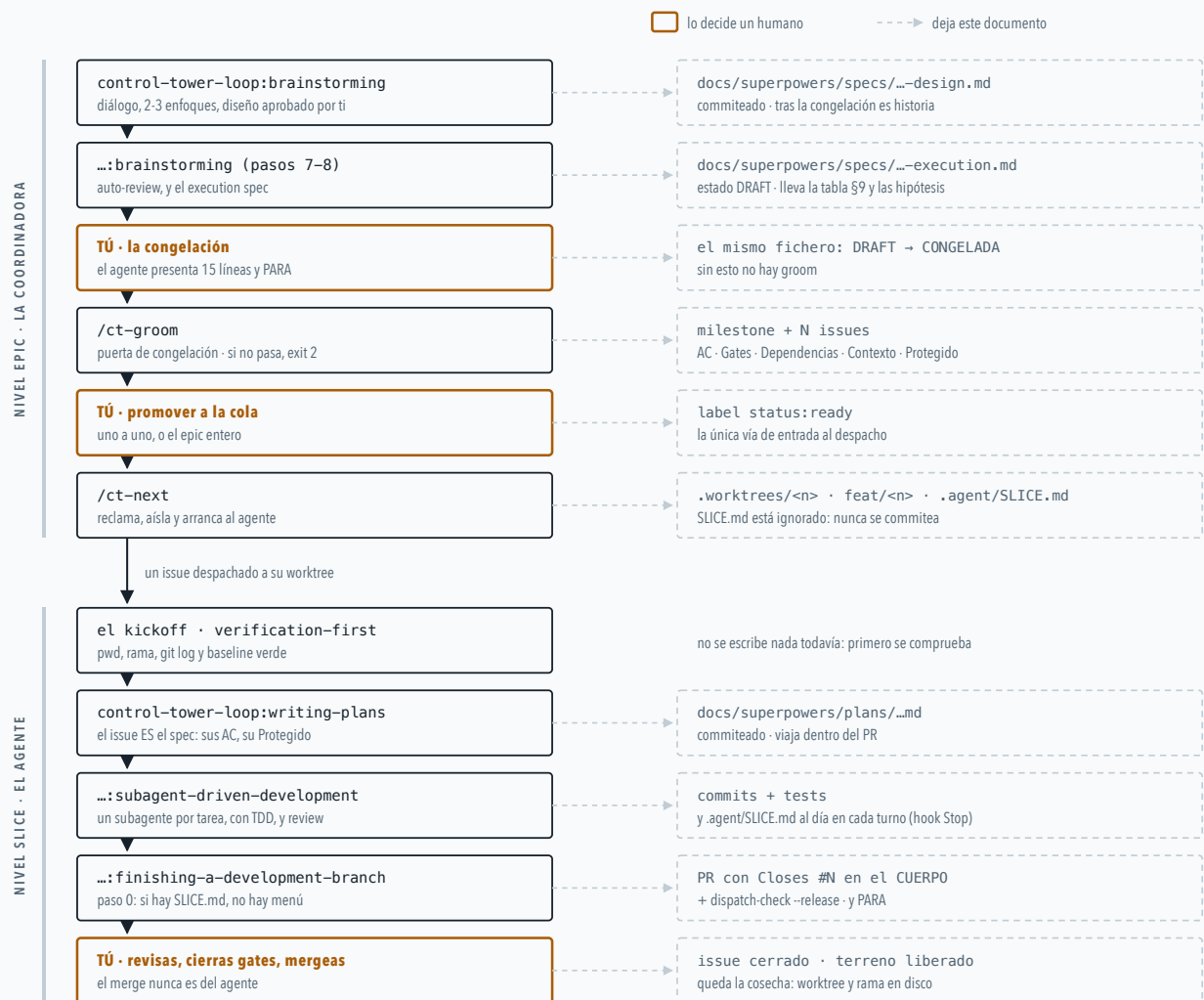
Ni una escritura en todo el camino, y hay un test que lo comprueba mirando con qué argumentos se llamó a `gh`. Por eso puedes invocarlo sin pensártelo.

DOS NIVELES

EL EPIC LO GOBIERNA CT; EL SLICE, LOS SKILLS

Control Tower no inventó cómo se trabaja *dentro* de un slice. Eso ya lo resolvía **superpowers**, el conjunto de skills de Jesse Vincent (MIT): pensar el diseño antes de escribir nada, TDD antes de implementar, verificar antes de declarar terminado. El plugin lleva un fork de **11 de esos skills** dentro — no los que parecían útiles, sino los que de verdad se usaban, medido sobre un barrido de 2.704 transcripts.

El reparto es limpio. **CT gobierna el nivel epic**: qué slices existen, cuál se despacha, quién reserva qué terreno. **Los skills gobiernan el nivel slice**: cómo se diseña, cómo se implementa, cómo se cierra. Cada nivel deja sus propios documentos por el camino, y casi todos acaban commiteados y viajando dentro del PR.



El carril de arriba se recorre **una vez por epic**, contigo delante; el de abajo lo recorre **cada agente despachado**, solo y en su worktree. Los tres pasos ámbar son las únicas decisiones que el loop no toma: congelar el spec, promover un slice a la cola, y mergear.

LAS TRES COSTURAS DEL FORK

El fork no es una copia: tres skills se reescribieron para que encajen con el loop, y hay un test que vigila que un cherry-pick futuro no las pise.

brainstorming ya no termina invocando `writing-plans`. Termina escribiendo el execution spec en `DRAFT` y **pidiendo la congelación**: un resumen de 15 líneas y tu OK. El design doc se queda como *handoff origen* y nadie lo vuelve a tocar.

subagent-driven-development, cuando no encuentra plan, ya no manda hacia atrás a brainstormear. Manda a escribir el plan *ahora*, acotado al issue — el diseño se congeló antes de que ese issue existiera.

finishing-a-development-branch traía un menú al terminar: mergear, abrir PR, limpiar la rama. Si existe `.agent/SLICE.md`, **no hay menú**: PR, `--release`, y parar.

LA PUERTA DE CONGELACIÓN

Antes incluso de mirar la tabla, `/ct-groom` hace dos comprobaciones sobre el spec entero. Un `[NEEDS CLARIFICATION]` sin resolver aborta: se admiten mientras el spec está en DRAFT, pero congelar con un hueco pendiente es inválido. Y una sección `## Hipótesis` ausente o vacía también aborta — **sin apuesta falsable no es un epic**, y el trabajo sin apuesta (mantenimiento, bugfixes) entra como issues sueltos, no por groom. Groom sólo mira que esté; la calidad de la hipótesis la juzgas tú al congelar.

LO QUE EL FORK TRAE Y NO APARECE EN EL CAMINO FELIZ

Los otros siete skills están ahí para cuando hacen falta: `test-driven-development`, `systematic-debugging`, `verification-before-completion`, `receiving-code-review`, `executing-plans`, `writing-skills` — y `using-git-worktrees`, que es el caso divertido: viaja en el fork y le diría al agente que se cree un worktree, así que **el kickoff se lo prohíbe explícitamente** («no crees worktrees nuevos, ya estás en el que te preparó el dispatcher»). Se descartaron los que CT vuelve innecesarios: `dispatching-parallel-agents`, cuyo hueco lo ocupa el propio despachador con aislamiento real, y `using-superpowers`, cuyo papel lo hace el plugin.

EL CERROJO

LAS LABELS DE GITHUB SON EL MUTEX

No hay base de datos ni demonio vigilando. Un slice está reservado porque su issue lleva puesta la label `status:in-progress`, y el terreno que toca está reservado porque lleva `area:api` o `touches:migration`. Dos slices que compartan un token no vuelan a la vez, y punto.

La sutileza está en que hay **dos recursos distintos con dos relojes distintos**: la plaza de agente concurrente (`--cap`) y el terreno. El agente termina, abre el PR y suelta su plaza — pero su trabajo sigue sin mergearse, así que el terreno sigue reservado.



Antes, abrir el PR soltaba las dos cosas a la vez — y el siguiente slice del mismo área ramificaba de una base que todavía no contenía ese trabajo. La ventana del cerrojo era más corta que la ventana del conflicto. Consecuencia práctica que hay que tener en la cabeza: **un PR sin mergearse frena a sus vecinos**.

ESTADO DEL ISSUE	QUÉ SIGNIFICA	OCUPA PLAZA	RESERVA TERRENO	QUIÉN LO PONE
status:backlog	groomeado, sin aprobar	NO	NO	/ct-groom
status:ready	aprobado, esperando turno	NO	NO	tú, a mano
status:in-progress	hay un agente dentro	SÍ	SÍ	/ct-next
status:in-review	PR abierto, sin mergear	NO	SÍ	el agente
cerrado	mergeado de verdad	NO	NO	tú

LOS GUARDARRAÍLES

TRES HOOKS QUE CORREN SOLOS

Un plugin de Claude Code puede engancharse a momentos concretos de la sesión. Control Tower usa tres, y cada uno tapa un agujero que se vio en campo:

Al arrancar la sesión — el agente se hidrata solo: lee el fichero de estado de su worktree y sabe quién es, qué slice lleva, cuál es su siguiente acción y si alguien lo declaró bloqueado. Sobrevive a un `/clear`.

Al cerrar cada turno — si hay commits por encima del último que el estado tiene apuntado, no te deja cerrar sin registrarlo. Es la puerta contra el agente que trabaja y se olvida de decir qué hizo.

Antes de cada comando de shell — la puerta del commit. Y esta tiene una historia.

EL COMMIT QUE CERRÓ UN ISSUE SIN QUERER

GitHub aplica las *closing keywords* de cualquier mensaje de commit que llegue a la rama por defecto. Y las comillas no protegen. En un repo real, un commit de **documentación** cuyo cuerpo contenía la cadena `Closes #451` —dentro de una frase que explicaba precisamente que el kickoff *no* la llevaba— cerró el issue #451 como completado.

Nadie quiso cerrar nada. Pero un slice que dependía de él pasó a tener su dependencia «satisfecha» sobre trabajo que no existía. Desde entonces hay un hook que **bloquea** ese commit — no avisa, bloquea: una comprobación cuyo resultado no puede detener la acción siguiente es decoración.

«ZSH: COMMAND NOT FOUND: LAUDE»

```
[oh-my-zsh] Would you like to update? [Y/n] laude --dangerously-skip...
```

```
zsh: command not found: laude
```

Primer despacho real contra producción: el dispatcher informó `exit 0`, «lanzados 1/1», «verificado». Lo que había en esa terminal era eso de arriba: el prompt de actualización de oh-my-zsh se comió la `c` mientras se tecleaba el comando. Veinte minutos de un issue reclamado **por nadie**, reteniendo el terreno del repo entero. Cero commits.

El fallo de fondo: se comprobaba el *continente* (existe una ventana con el título correcto) en vez del *contenido* (el comando llegó a ejecutarse). Ahora el agente escribe un centinela en disco al arrancar, y si el centinela no aparece, el dispatcher reintenta — y si aun así no aparece, **no dice «lanzado»**.

HONESTIDAD

LO QUE EL LOOP NO TE GARANTIZA

Esta es la parte que más me gusta contar, porque está escrita dentro del propio plugin en vez de barrida debajo de la alfombra:

EL CLAIM NO ES ATÓMICO

Reclamar un issue son labels de GitHub, sin compare-and-swap. Está reproducido —no sospechado— que dos despachadores lanzados casi a la vez reclaman el mismo terreno y arrancan los dos. La mitigación de hoy es operativa: **no lances dos `/ct-next` a la vez contra el mismo repo**.

LOS GATES SE ENSEÑAN

El loop escribe en el issue y en el prompt del agente qué hay que revisar antes de mergear —una captura, un dry-run aprobado—, pero no impide el merge. El que cierra el gate eres tú.

LA EVIDENCIA ES LOCAL

«Sin señal de vida» significa «en esta máquina no hay ningún proceso trabajando ahí». Nunca «está muerto». Y si la herramienta que lo comprueba falla, el informe dice *no se pudo comprobar* y sale con error — jamás acusa de abandono a un slice sano.

NADA VIGILA ENTRE CORRIDAS

El claim es una label, sin latido. Un agente que muere deja su reserva puesta hasta que alguien corra `/ct-status` o `/ct-next` y se dé cuenta.

La regla que atraviesa todo el plugin: si no se puede confirmar que el agente arrancó, el resultado *no puede ser* «lanzado» con exit 0.

control-tower-loop v0.32.0 · plugin de Claude Code · MIT

Versión viva, con los diagramas ampliables: <https://claude.ai/code/artifact/c4339a41-40ee-4d4b-9097-0c7604d9da24>